Open in app ↗

Medium Search Play Games Write 3

Welcome Offer Access to everything. Now 30% off. Upgrade now

Databricks + Data Engineering + Spark + Python + Big Data +

Databricks Level 5 (Production Readiness & Observability) — Building Observable, Resilient, Self-Healing Data Systems That You Trust Completely



Anurag Sharma 17 min read · Oct 19, 2026

4



Summary

Databricks Level 5 (Production Readiness & Observability) is where your data pipeline stops being something you nervously check at 3 AM and becomes a system you genuinely trust. If Databricks Level 4 (Pipeline Engineering) taught you how to orchestrate and run pipelines reliably, Databricks Level 5 teaches you how to *know* they are running well, predict failures before they happen, isolate issues instantly, and build resilience into every layer. This blog covers comprehensive observability frameworks, golden signals for data systems, anomaly detection, SLA forecasting, advanced alerting strategies, incident prediction, chaos engineering for data platforms, distributed tracing across pipeline dependencies, metrics standardization, and the operational maturity that separates “we have monitoring” from “we run production systems with confidence.” By the end, you will understand how to build data systems that are not just reliable but *provably* reliable.

Who Should Read This Blog

1. **Data engineers running pipelines that have reached production criticality**
2. **Tech leads managing teams where “data is down” incidents are expensive**
3. **Anyone preparing for principal or staff-level data engineering interviews**
4. **Platform architects designing multi-team data infrastructures**
5. **Engineers who have experienced preventable production incidents**
6. **Teams with SLA commitments to business stakeholders**
7. **Anyone transitioning from “job queue” to “data platform” mindset**

The Difference Between “Running” and “Production Ready”

> *There’s a specific moment in every data engineering organization where someone says:*
>
> *“Why don’t we know when things break until someone files a ticket?”*
>
> *Or worse:*
>
> *“The dashboard showed yesterday’s data for six hours before anyone noticed.”*
>
> *Or the worst:*
>
> *“We double-counted revenue for three weeks.”*

> *These are not orchestration problems. Databricks Level 4 (Pipeline Engineering) would have caught most of them.*
>
> *These are observability problems.*
>
> *Your pipeline ran. It succeeded. But you had no idea it was producing wrong data until it was too late.*

The Three Pillars of Production-Ready Systems

A system is truly production-ready when it has:

1. **Observability** — You can answer any question about what happened, when, and why
2. **Resilience** — It can withstand failures and recover without human intervention
3. **Predictability** — You know when things will break before they actually do

Most teams think Databricks Level 4 (Pipeline Engineering) is enough.

It is not.

Databricks Level 4 makes pipelines *run*. Databricks Level 5 makes pipelines *trustworthy*.

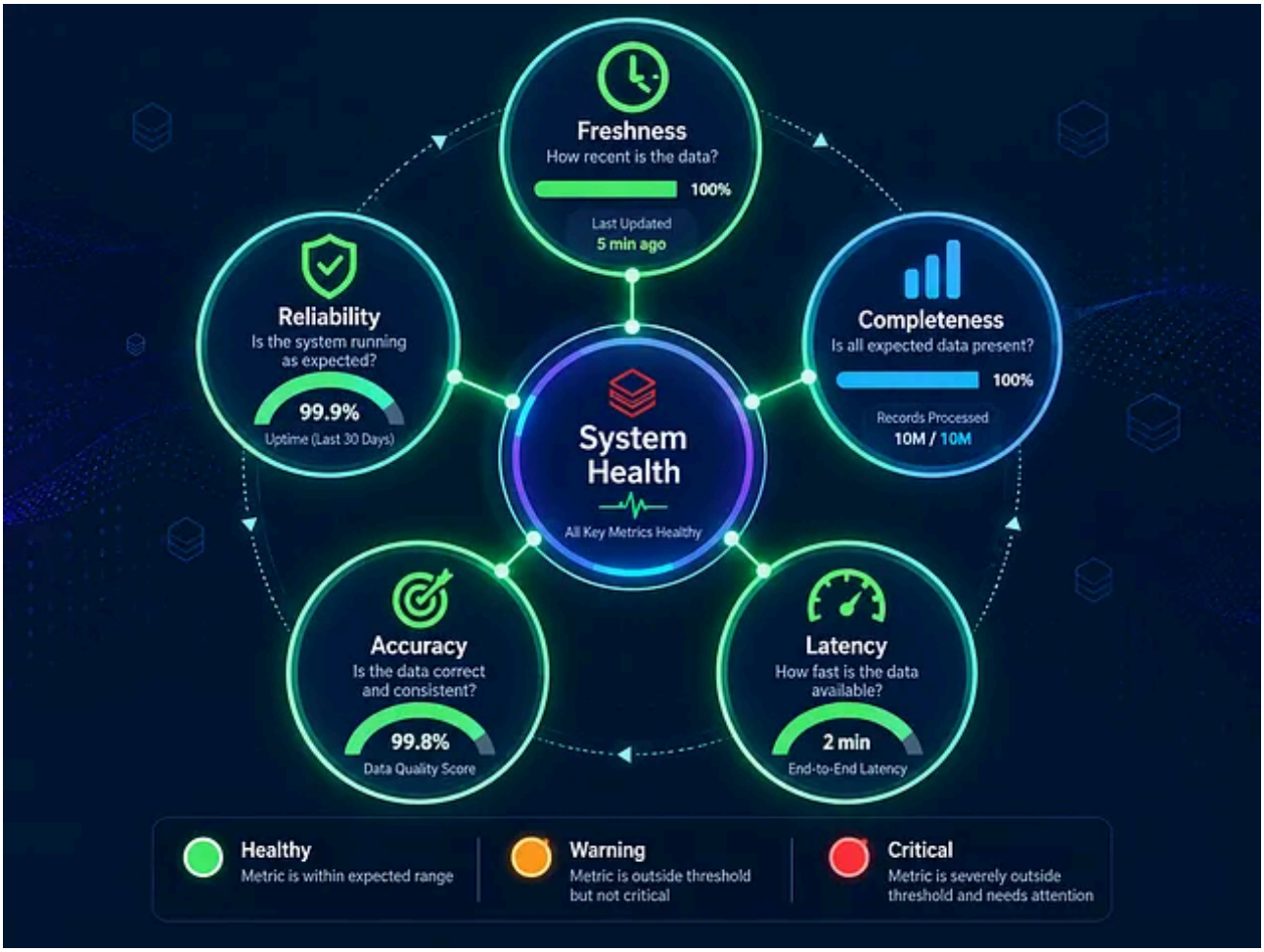
Step 1: The Golden Signals Framework for Data Systems

Google's SRE book introduced the concept of "golden signals" — a small set of metrics that, if you measure nothing else, tell you everything you need to know about system health.

The four golden signals are:

- **Latency** — how long requests take
- **Traffic** — how many requests you serve
- **Errors** — how many requests fail
- **Saturation** — how full your system is

These work for web services. Data systems need a parallel framework.



The five golden signals for data pipelines

Signal 1: Freshness

How recent is the data?

```
freshness_hours = (datetime.now() - last_successful_run).total_seconds()/3600
acceptable_freshness_hours = 6

if freshness_hours > acceptable_freshness_hours:
    # SLA BREACH
    severity = "CRITICAL"
```

Why it matters: A dashboard showing yesterday's data when it should show today's is worse than a dashboard showing nothing.

Signal 2: Completeness

Did all expected data arrive?

```
expected_record_count = 100000
actual_record_count = spark.table("silver.orders").count()
completeness_percentage = (actual_record_count/expected_record_count) * 100

if completeness_percentage < 95:
    # INCOMPLETE
    severity = "WARNING"
```

Why it matters: Missing data is silent data loss. You might never know.

Signal 3: Latency

How long do pipelines take?

```
pipeline_duration_seconds = (run_end_time - run_start_time).total_seconds()
acceptable_latency_seconds = 1800  # 30 minutes

if pipeline_duration_seconds > acceptable_latency_seconds:
    # SLOW
    severity = "WARNING"
```

Why it matters: Increasing latency often signals resource constraints before they cause failures.

Signal 4: Accuracy

Does the data make business sense?

```
total_revenue = spark.sql("SELECT SUM(amount) FROM gold.daily_revenue").collect(
expected_revenue_range = (1000000, 5000000)

if not (expected_revenue_range[0] <= total_revenue <= expected_revenue_range[1])
    # ANOMALY
    severity = "CRITICAL"
```

Why it matters: A pipeline can run successfully and produce nonsensical data.

Signal 5: Reliability

Is the pipeline succeeding consistently?

```
success_rate = (successful_runs / total_runs) * 100
acceptable_success_rate = 99.5

if success_rate < acceptable_success_rate:
```

```
    # RELIABILITY DEGRADED
    severity = "WARNING"
```

Why it matters: Flaky pipelines that sometimes work but sometimes don't are harder to debug than consistently broken pipelines.

Implementing the five golden signals

Create a monitoring table that tracks all five:

```
from datetime import datetime

signal_record = {
    "pipeline_name": "daily_revenue",
    "run_id": "862645",
    "timestamp": datetime.now(),
    "freshness_hours": 2.5,
    "freshness_ok": True,
    "completeness_pct": 99.8,
    "completeness_ok": True,
    "latency_seconds": 1200,
    "latency_ok": True,
    "accuracy_ok": True,
    "reliability_pct": 99.7,
    "reliability_ok": True,
    "overall_health": "HEALTHY"
}

( spark.createDataFrame([signal_record])
 .write
 .mode("append")
 .saveAsTable("monitoring.golden_signals") )
```

Then query it to understand system health:

```
SELECT
  DATE(timestamp) AS date,
  pipeline_name,
```

```
  AVG(freshness_hours) AS avg_freshness,
  SUM(CASE WHEN freshness_ok THEN 1 ELSE 0 END) / COUNT(*) AS freshness_ok_pct,
  SUM(CASE WHEN completeness_ok THEN 1 ELSE 0 END) / COUNT(*) AS completeness_ok
  SUM(CASE WHEN latency_ok THEN 1 ELSE 0 END) / COUNT(*) AS latency_ok_pct,
  SUM(CASE WHEN accuracy_ok THEN 1 ELSE 0 END) / COUNT(*) AS accuracy_ok_pct,
  SUM(CASE WHEN reliability_ok THEN 1 ELSE 0 END) / COUNT(*) AS reliability_ok_p
FROM monitoring.golden_signals
GROUP BY DATE(timestamp), pipeline_name
ORDER BY date DESC
```

If all five golden signals are green, your system is healthy. If any are red, investigate immediately.

Step 2: Observability vs Monitoring — Understanding the Difference

Many teams conflate these. They are not the same.

Monitoring is *measurement*.
You have dashboards. You collect metrics. You alert on thresholds.

Observability is *understanding*.
You can explain *why* something happened, trace an issue to its root cause, and predict what will happen next.

Example of monitoring but not observability

```
alerts:
  - name: "Pipeline Failure Alert"
    condition: "job_status == FAILED"
    notify: "data-team@company.com"
```

This is monitoring. You know when something fails.

But it is not observing. You don't know *why* it failed.

Was it:

- missing source data?
- schema change?
- resource exhaustion?
- downstream dependency failure?
- logic error?

Example of true observability

```
# Log structured context at every step
logging_context = {
    "pipeline": "daily_revenue",
    "run_id": run_id,
    "stage": "ingestion",
    "source_table": "bronze.orders",
    "timestamp": datetime.now(),
    "source_row_count": source_df.count(),
    "expected_min_rows": 10000,
    "actual_min_rows": source_df.count(),
    "checks_passed": source_df.count() >= 10000,
}

# Structured logging lets you query later
log_to_monitoring_table(logging_context)

if source_df.count() < 10000:
    raise Exception(f"Ingestion check failed: {logging_context}")
```

Now if ingestion fails, you can query:

```
SELECT *
FROM monitoring.pipeline_logs
WHERE pipeline = "daily_revenue"
AND stage = "ingestion"
AND checks_passed = false
ORDER BY timestamp DESC
LIMIT 10
```

You instantly see: “Source had only 5,000 rows instead of expected 10,000.”

That is observability.

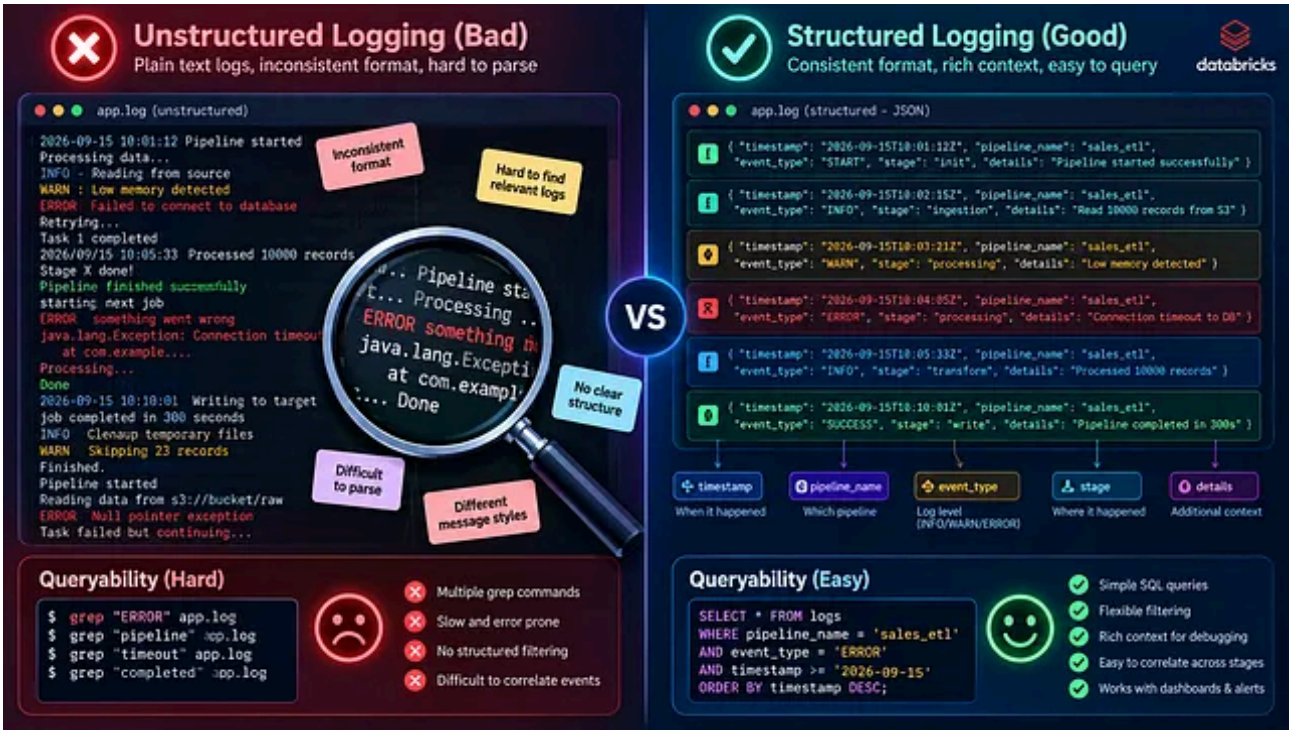
Step 3: Structured Logging — The Foundation of Observability

Unstructured logging is noise. Structured logging is signal.

Bad logging

```
print("Pipeline started")
print("Processing orders")
print("Done")
```

This tells you nothing when you query logs three weeks later.



Good structured logging

```
import json
from datetime import datetime

def log_event(event_type, stage, details):
    log_record = {
        "timestamp": datetime.now().isoformat(),
        "pipeline_name": "daily_revenue",
        "run_id": dbutils.entry_point.get_job_run_id(),
        "event_type": event_type,  # "START", "PROGRESS", "ERROR", "SUCCESS"
        "stage": stage,
        "details": details
    }

    # Write to monitoring table, not stdout
    spark.createDataFrame([log_record]).write.mode("append").saveAsTable("monito

# Usage
log_event("START", "ingestion", {"source": "bronze.orders"})

source_count = spark.table("bronze.orders").count()
log_event("PROGRESS", "ingestion", {
    "source": "bronze.orders",
    "row_count": source_count,
    "timestamp": datetime.now().isoformat()
})
```

```
transformed_count = result_df.count()
log_event("PROGRESS", "transformation", {
    "source_rows": source_count,
    "result_rows": transformed_count,
    "dropped_rows": source_count - transformed_count
})

log_event("SUCCESS", "pipeline", {"rows_written": transformed_count})
```

Now you can query events and reconstruct exactly what happened:

```
SELECT *
FROM monitoring.events
WHERE run_id = '862645'
ORDER BY timestamp ASC
```

Output:

```
timestamp               | event_type | stage          | details
2026-10-15T02:00:00     | START      | ingestion      | source: bronze.orders
2026-10-15T02:01:23     | PROGRESS   | ingestion      | row_count: 150000
2026-10-15T02:05:45     | PROGRESS   | transformation | source_rows: 150000,
2026-10-15T02:07:12     | SUCCESS    | pipeline       | rows_written: 149500
```

That tells a complete story.

Structured logging best practices

1. Always include context

- pipeline name
- run ID

- stage name
- timestamp

2. Log at decision points

- validation checks
- branching logic
- error conditions

3. Log metrics that matter

- row counts before/after
- processing time per stage
- resource utilization

4. Use consistent schemas

- same fields across all logs
- makes querying and alerting easier

Step 4: Anomaly Detection — Letting the Machine Spot Problems Before You Do

As your system matures, you can't manually threshold every metric.

At some point, you need the system to detect its own anomalies.

Simple statistical anomaly detection

Compare today's metric against its historical distribution:

```
from datetime import datetime, timedelta

pipeline_name = "daily_revenue"
metric_name = "pipeline_latency_seconds"

# Get historical data (last 30 days)
historical_df = spark.sql(f"""
    SELECT {metric_name}
    FROM monitoring.metrics
    WHERE pipeline_name = '{pipeline_name}'
    AND timestamp >= CURRENT_DATE() - 30
    AND timestamp < CURRENT_DATE()
""")

historical_values = [row[0] for row in historical_df.collect()]

# Calculate statistics
import statistics
mean = statistics.mean(historical_values)
stdev = statistics.stdev(historical_values)
upper_threshold = mean + (3 * stdev)  # 3-sigma rule
lower_threshold = mean - (3 * stdev)

# Check today's metric
today_df = spark.sql(f"""
    SELECT {metric_name}
    FROM monitoring.metrics
    WHERE pipeline_name = '{pipeline_name}'
    AND DATE(timestamp) = CURRENT_DATE()
""")

today_values = [row[0] for row in today_df.collect()]

for value in today_values:
    if value > upper_threshold or value < lower_threshold:
        # Anomaly detected!
        anomaly_record = {
            "pipeline": pipeline_name,
            "metric": metric_name,
            "value": value,
            "expected_range": f"{lower_threshold:.0f} - {upper_threshold:.0f}",
            "severity": "WARNING",
            "timestamp": datetime.now()
        }
        spark.createDataFrame([anomaly_record]).write.mode("append").saveAsTable
```

More sophisticated: exponential weighted moving average (EWMA)

Recent data points should matter more than old ones:

```
def calculate_ewma(values, alpha=0.3):
    """
    alpha = 0.3 means 30% weight on current, 70% on history
    alpha = 0.7 means 70% weight on current, 30% on history
    """
    ewma = values[0]
    for value in values[1:]:
        ewma = (alpha * value) + ((1 - alpha) * ewma)
    return ewma

# Use EWMA to detect trend changes
historical_ewma = calculate_ewma(historical_values, alpha=0.2)
current_value = today_values[-1]

percent_change = ((current_value - historical_ewma) / historical_ewma) * 100

if abs(percent_change) > 20:  # 20% deviation
    # Anomaly!
    send_alert(f"Pipeline latency anomaly: {percent_change:.1f}% change")
```

Even more sophisticated: isolation forest or other ML methods

For truly complex patterns, use unsupervised learning:

```
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.clustering import IsolationForest

# Prepare training data
training_df = spark.sql("""
    SELECT
        pipeline_latency_seconds,
        row_count,
        error_count,
        retry_count
    FROM monitoring.metrics
    WHERE timestamp >= CURRENT_DATE() - 90
""")

# Train isolation forest
```

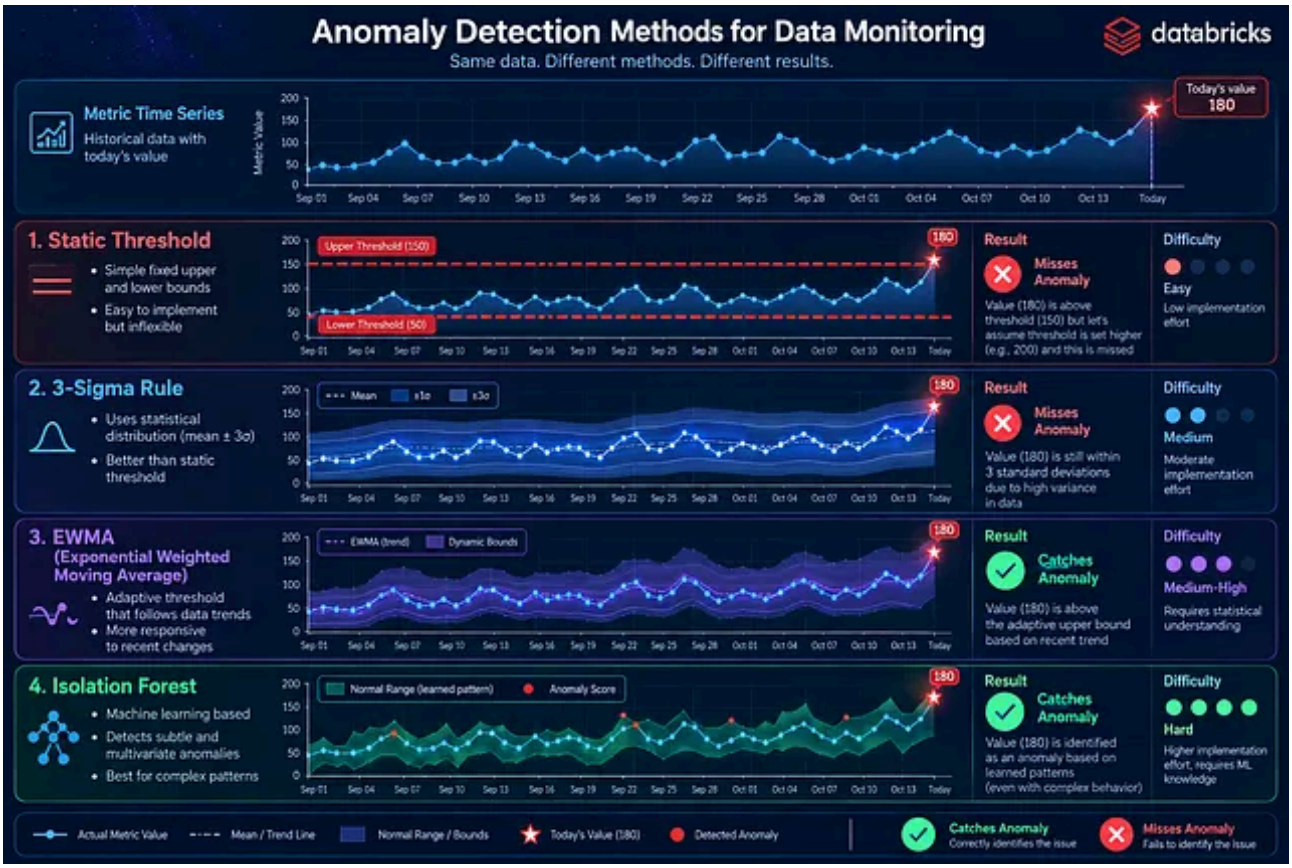
```
assembler = VectorAssembler(inputCols=["pipeline_latency_seconds", "row_count",
training_features = assembler.transform(training_df)

iso_forest = IsolationForest(contamination=0.05, numTrees=100)
model = iso_forest.fit(training_features)

# Score today's data
today_features = assembler.transform(today_df)
predictions = model.transform(today_features)

# predictions column has: -1 (anomaly), 0 (normal)
anomalies = predictions.filter("predictions == -1")

if anomalies.count() > 0:
    send_alert("Anomaly detected in today's pipeline metrics")
```



When to use which

Method	When to use	Pros	Cons
Static thresholds	Simple metrics, clear bounds	Easy to understand	Not adaptive
3-sigma rule	Normally distributed metrics	Statistical rigor	Needs history
EWMA	Trending metrics	Responsive to recent changes	Requires tuning
Isolation forest	Complex multivariate patterns	Detects subtle combinations	Needs ML overhead

Start with static thresholds. Graduate to statistical methods. Use ML only if justified.

Step 5: SLA Forecasting — Predicting Breaches Before They Happen

The holy grail of observability: know your system will fail *before* it fails.

Simple breach prediction

```
from datetime import datetime, timedelta

pipeline_name = "daily_revenue"
sla_freshness_hours = 6

# Check current freshness
last_successful_run = spark.sql(f"""
    SELECT MAX(timestamp)
    FROM monitoring.pipeline_runs
    WHERE pipeline_name = '{pipeline_name}'
    AND status = 'SUCCESS'
""").collect()[0][0]

hours_since_last_run = (datetime.now() - last_successful_run).total_seconds() /

# If we're approaching SLA, forecast breach
hours_until_breach = sla_freshness_hours - hours_since_last_run
```

```
    if hours_until_breach < 1:  # Less than 1 hour to SLA
        send_warning_alert(f"SLA breach imminent for {pipeline_name}: {hours_until_b
```

Predictive breach forecasting

Predict whether today's pipeline will breach SLA based on current progress:

```
# Get today's pipeline run
current_run = spark.sql(f"""
    SELECT
        timestamp AS run_start,
        status
    FROM monitoring.pipeline_runs
    WHERE pipeline_name = '{pipeline_name}'
    AND DATE(timestamp) = CURRENT_DATE()
    ORDER BY timestamp DESC
    LIMIT 1
""").collect()[0]

# Get historical runtimes for this pipeline
historical_durations = spark.sql(f"""
    SELECT
        (end_time - start_time) / 3600.0 AS duration_hours
    FROM monitoring.pipeline_runs
    WHERE pipeline_name = '{pipeline_name}'
    AND status = 'SUCCESS'
    AND DATE(start_time) >= CURRENT_DATE() - 30
""")

# Calculate percentiles
percentiles_df = historical_durations.selectExpr(
    "percentile_approx(duration_hours, 0.5) AS p50",
    "percentile_approx(duration_hours, 0.95) AS p95"
)

p50, p95 = percentiles_df.collect()[0]

# Predict completion time
elapsed_since_start = (datetime.now() - current_run["run_start"]).total_seconds(
predicted_completion = current_run["run_start"] + timedelta(hours=p95)

# Check against SLA
sla_deadline = current_run["run_start"] + timedelta(hours=sla_freshness_hours)

if predicted_completion > sla_deadline:
    # Will breach!
```

```
        send_alert(f"SLA breach PREDICTED: {pipeline_name} will likely miss SLA dead
        send_alert(f"Predicted: {predicted_completion}, SLA: {sla_deadline}")
```

Root cause analysis for predicted breaches

Don't just predict failure. Predict *why* it will fail:

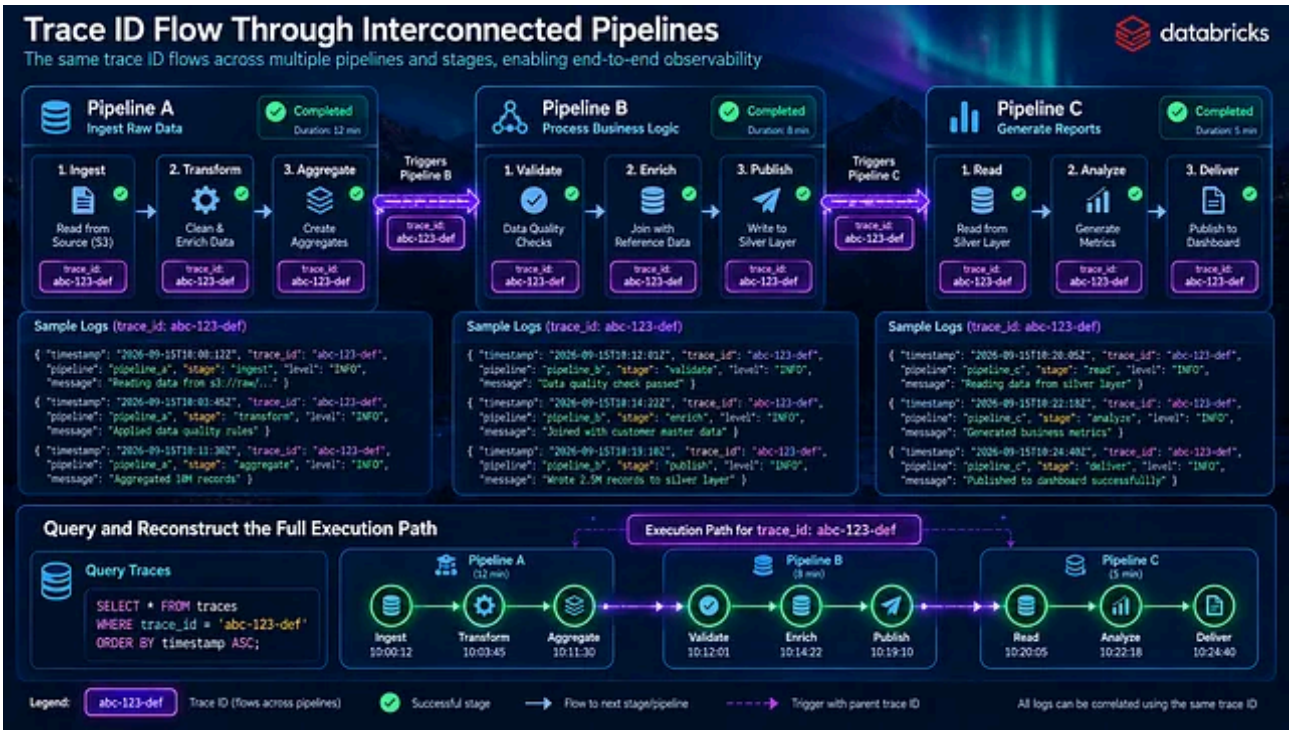
```
# Analyze historical failures
failure_causes = spark.sql(f"""
    SELECT
        error_type,
        COUNT(*) AS frequency,
        AVG(retry_count) AS avg_retries
    FROM monitoring.events
    WHERE pipeline_name = '{pipeline_name}'
    AND event_type = 'ERROR'
    AND timestamp >= CURRENT_DATE() - 30
    GROUP BY error_type
    ORDER BY frequency DESC
""")

# If we see a pattern emerging, alert on it
for row in failure_causes.collect():
    if row["frequency"] > 3:  # Seeing this error frequently
        send_alert(f"Recurring error in {pipeline_name}: {row['error_type']} (se
```

Step 6: Distributed Tracing Across Pipeline Dependencies

When Pipeline A fails, Pipeline B doesn't run, and Pipeline C gets stale data, how do you trace the issue?

Distributed tracing lets you follow a logical request through multiple services/jobs.



Trace ID propagation

```
import uuid
from datetime import datetime

# Generate trace ID at pipeline start
trace_id = str(uuid.uuid4())

def run_pipeline_stage(stage_name):
    stage_log = {
        "trace_id": trace_id,
        "pipeline": "daily_analytics",
        "stage": stage_name,
        "timestamp": datetime.now(),
        "duration_ms": None
    }

    start_time = datetime.now()

    try:
        if stage_name == "ingest":
            # run ingestion
            pass
        elif stage_name == "transform":
            # run transformation
            pass
        elif stage_name == "aggregate":
            # run aggregation
```

```
            pass

        stage_log["status"] = "SUCCESS"
    except Exception as e:
        stage_log["status"] = "FAILED"
        stage_log["error"] = str(e)
    finally:
        stage_log["duration_ms"] = (datetime.now() - start_time).total_seconds()
        spark.createDataFrame([stage_log]).write.mode("append").saveAsTable("mon

# Run pipeline stages
for stage in ["ingest", "transform", "aggregate"]:
    run_pipeline_stage(stage)
```

Now you can query a complete trace:

```
SELECT *
FROM monitoring.traces
WHERE trace_id = 'abc-123-def'
ORDER BY timestamp ASC
```

Output shows the full journey of this pipeline execution.

Cross-pipeline tracing

When Pipeline A triggers Pipeline B, pass the trace ID:

```
# Pipeline A
trace_id = str(uuid.uuid4())

# Do work in Pipeline A
spark.createDataFrame([{"trace_id": trace_id, ...}]).write.saveAsTable("monitori

# Trigger Pipeline B with trace ID
trigger_downstream_pipeline(
    pipeline_name="daily_reporting",
    parameters={"parent_trace_id": trace_id}
)
```

```
# --- Now in Pipeline B ---
parent_trace_id = dbutils.widgets.get("parent_trace_id")

pipeline_b_log = {
    "trace_id": parent_trace_id,  # Same trace ID!
    "pipeline": "daily_reporting",
    "stage": "ingestion",
    "timestamp": datetime.now()
}
```

Now a single trace ID connects Pipeline A → Pipeline B → Pipeline C.



When something breaks, you can see the full dependency chain of who triggered what.

Step 7: Comprehensive Health Dashboards

All the monitoring in the world is useless if nobody looks at it.

Your observability system needs good dashboards.

The operations dashboard

Designed for on-call engineers. Shows:

- Pipeline status summary (green/red/yellow)
- Last 24 hours of runs with timing

- Failure count and retry count
- Alert history
- Recent anomalies

The reliability dashboard

Designed for leadership. Shows:

- SLA compliance % over time
- MTTR (mean time to recovery)
- Incident count and severity
- Data freshness trends
- Cost trends

The business dashboard

Designed for data consumers. Shows:

- Are the dashboards up to date?
- When was the last update?
- Is data complete?
- Known issues and ETAs

Example: health dashboard query

```
WITH recent_runs AS (
    SELECT
        pipeline_name,
        MAX(run_start) AS last_run_start,
        MAX(CASE WHEN status = 'SUCCESS' THEN run_end ELSE NULL END) AS last_suc
        SUM(CASE WHEN status = 'FAILED' THEN 1 ELSE 0 END) AS failure_count,
        COUNT(*) AS total_runs,
```

```
        AVG(CASE WHEN status = 'SUCCESS' THEN (run_end - run_start) / 3600.0 ELS
    FROM monitoring.pipeline_runs
    WHERE timestamp >= CURRENT_DATE() - 1
    GROUP BY pipeline_name
)

SELECT
    pipeline_name,
    CASE
        WHEN (CURRENT_TIMESTAMP() - last_success_end) / 3600.0 > 6 THEN 'RED'
        WHEN (CURRENT_TIMESTAMP() - last_success_end) / 3600.0 > 4 THEN 'YELLOW'
        ELSE 'GREEN'
    END AS status,
    failure_count,
    total_runs,
    (failure_count / total_runs) * 100 AS failure_pct,
    avg_success_duration_hours,
    last_run_start
FROM recent_runs
ORDER BY status DESC, failure_pct DESC
```

Step 8: Incident Response and Runbooks — Operationalizing Recovery

Observability without action is just noise.

When an alert fires, you need a runbook.

Runbook structure

```
# Pipeline: daily_revenue

## Symptom
Pipeline fails to complete by 6 AM, or data is incomplete.

## Immediate Actions (< 5 min)
1. Check Databricks Workflows console for error message
2. Check monitoring.events table for last successful stage
3. Verify source data is available in bronze.orders

## Diagnosis (< 15 min)
- If error is "schema mismatch":
  - Query bronze.orders schema: `DESCRIBE bronze.orders`
```

```
  - Compare to expected schema in documentation
  - If new columns: expected, run with `mergeSchema=true`

- If error is "insufficient data":
  - Check source row count: `SELECT COUNT(*) FROM bronze.orders WHERE _ingest_da
  - If < 10k rows: contact source system team

- If error is "timeout":
  - Pipeline took too long, likely hitting cluster limits
  - Check cluster metrics: CPU, memory, disk

## Recovery Steps
1. If schema issue: update Auto Loader config, rerun
2. If data issue: wait for source data, then rerun
3. If timeout: increase cluster size, submit new run
4. If unknown: escalate to data platform team

## Follow-up
1. Document what went wrong
2. Create Jira ticket for permanent fix
3. Update monitoring thresholds if needed
```

Runbook automation

Auto-execute recovery steps when possible:

```
def auto_recovery_handler(error_type, pipeline_name):
    """Attempt automatic recovery based on error type"""

    if error_type == "SCHEMA_MISMATCH":
        # Auto-recovery: update Auto Loader with mergeSchema
        update_auto_loader_config(pipeline_name, merge_schema=True)
        trigger_rerun(pipeline_name)
        send_alert(f"Auto-recovered {pipeline_name}: schema mismatch fixed and r

    elif error_type == "INSUFFICIENT_DATA":
        # No auto-recovery: requires manual intervention
        send_alert(f"Manual intervention required: {pipeline_name} has insuffici
        create_jira_ticket(f"Missing data for {pipeline_name}")

    elif error_type == "TIMEOUT":
        # Auto-recovery: increase cluster size and rerun
        scale_cluster(pipeline_name, scale_factor=1.5)
        trigger_rerun(pipeline_name)
        send_alert(f"Auto-recovered {pipeline_name}: scaled cluster and rerun tr
```

```
    else:
        # Unknown: escalate
        send_critical_alert(f"Unknown error in {pipeline_name}: {error_type}")
        page_oncall_engineer()
```

Step 9: Chaos Engineering for Data Systems

You want to know your system is resilient *before* a real failure happens.

Chaos engineering deliberately injects failures and observes how your system responds.

Chaos experiment 1: Missing source data

```
"""
Hypothesis: If source data doesn't arrive, pipeline fails gracefully and alerts.
Experiment: Rename source directory, run pipeline.
Expected: Pipeline fails with clear error, alert sent.
"""

import os
from datetime import datetime

# Move source data
source_path = "abfss://landing@storage/orders/"
backup_path = "abfss://landing-backup@storage/orders_chaos_backup/"

move_directory(source_path, backup_path)

try:
    # Run pipeline
    trigger_pipeline("daily_revenue")

    # Wait for completion
    time.sleep(120)

    # Check if alert was sent
    alerts = spark.sql("""
        SELECT * FROM monitoring.alerts
        WHERE timestamp >= CURRENT_TIMESTAMP() - INTERVAL 3 MINUTES
        AND alert_type = 'SOURCE_DATA_MISSING'
    """).collect()
```

```
    if len(alerts) > 0:
        print("✓ CHAOS TEST PASSED: Alert sent correctly for missing source data
    else:
        print("✗ CHAOS TEST FAILED: No alert sent for missing source data")

finally:
    # Restore source data
    move_directory(backup_path, source_path)
```

Chaos experiment 2: Slow downstream dependency

```
"""
Hypothesis: If a downstream pipeline is slow, upstream pipelines don't wait fore
Experiment: Make a downstream job slow, see if upstream times out correctly.
"""

# Simulate slow downstream by adding delay
add_delay_to_pipeline("daily_reporting", delay_seconds=3600)

try:
    # Trigger upstream
    trigger_pipeline("daily_revenue")

    # Should complete within normal time, not wait for downstream
    run_duration = monitor_pipeline_duration("daily_revenue", timeout_seconds=30

    if run_duration < 300:
        print("✓ CHAOS TEST PASSED: Upstream didn't wait for slow downstream")
    else:
        print("✗ CHAOS TEST FAILED: Upstream waited for downstream")

finally:
    # Remove delay
    remove_delay_from_pipeline("daily_reporting")
```

Chaos experiment 3: Data quality degradation

```
"""
Hypothesis: If source data quality drops, we detect it and don't corrupt Gold la
```

```
    Experiment: Inject bad data into Bronze (negative amounts, nulls, duplicates).
    """

    # Inject bad data
    bad_data = spark.createDataFrame([
        {"order_id": "O1", "amount": -100},  # negative amount
        {"order_id": "O2", "amount": None},   # null amount
        {"order_id": "O3", "amount": 50},
        {"order_id": "O3", "amount": 50},     # duplicate
    ]).toDF("order_id", "amount")

    bad_data.write.mode("append").saveAsTable("bronze.orders_chaos")

    try:
        # Run transformation
        trigger_pipeline("silver_orders")

        # Check if bad records were quarantined
        invalid_records = spark.sql("""
            SELECT * FROM monitoring.invalid_records
            WHERE source_table = 'bronze.orders_chaos'
            AND timestamp >= CURRENT_TIMESTAMP() - INTERVAL 5 MINUTES
        """).collect()

        if len(invalid_records) >= 2:
            print("✓ CHAOS TEST PASSED: Bad data was detected and quarantined")
        else:
            print("✗ CHAOS TEST FAILED: Bad data not caught")

    finally:
        # Cleanup
        spark.sql("DROP TABLE bronze.orders_chaos")
```

Chaos experiment schedule

Run chaos experiments periodically:

```
# Chaos testing schedule
chaos_tests:
  - name: "missing_source_data"
    frequency: "weekly"
    day: "Thursday"
    time: "4 PM"

  - name: "slow_downstream"
    frequency: "weekly"
```

```
      day: "Friday"
      time: "2 PM"

    - name: "data_quality_degradation"
      frequency: "bi-weekly"
      day: "Tuesday"
      time: "3 PM"
```

This forces you to discover and fix issues before customers do.

Step 10: Metrics Standardization — Making Observability Reusable

If every team defines metrics differently, comparing systems is impossible.

Establish standards.

Metric naming conventions

Use a consistent naming scheme:

```
{system}_{component}_{metric_type}_{unit}

Examples:
- pipeline_ingestion_duration_seconds
- pipeline_transform_row_count_total
- pipeline_gold_error_rate_percent
- cluster_utilization_percent
- cluster_cost_usd_hourly
```

Common metric definitions

Define once, reuse everywhere:

```
# Metrics registry
METRICS = {
    "pipeline_duration": {
        "description": "Time from start to completion",
        "unit": "seconds",
        "measurement": "end_time - start_time",
        "threshold_warning": 1800,  # 30 minutes
        "threshold_critical": 3600,  # 1 hour
    },
    "row_completeness": {
        "description": "Percentage of expected rows received",
        "unit": "percent",
        "measurement": "(actual_count / expected_count) * 100",
        "threshold_warning": 95,
        "threshold_critical": 90,
    },
    "error_rate": {
        "description": "Percentage of pipeline runs that failed",
        "unit": "percent",
        "measurement": "(failed_runs / total_runs) * 100",
        "threshold_warning": 2,
        "threshold_critical": 5,
    },
}
```

Step 11: Cost Observability — Knowing What Your Production System Actually Costs

Level 5 (Production Readiness & Observability) includes cost as a key metric.

Track cost per pipeline

```
cluster_config = {
    "spark_version": "15.2.x-scala2.12",
    "node_type": "i3.xlarge",
    "num_workers": 4,
    "aws_attributes": {"availability": "SPOT"}
}

# i3.xlarge on SPOT: ~$0.40/hour
hourly_cost = 0.40 * (1 + 4)  # driver + workers
run_duration_hours = pipeline_duration_seconds / 3600
run_cost = hourly_cost * run_duration_hours
```

```
cost_record = {
    "pipeline": "daily_revenue",
    "run_id": run_id,
    "cluster_config": cluster_config,
    "duration_seconds": pipeline_duration_seconds,
    "cost_usd": run_cost,
    "timestamp": datetime.now()
}

spark.createDataFrame([cost_record]).write.mode("append").saveAsTable("monitorin
```

Then analyze cost trends:

```
SELECT
    DATE(timestamp) AS date,
    pipeline,
    COUNT(*) AS run_count,
    SUM(cost_usd) AS daily_cost,
    AVG(cost_usd) AS avg_cost_per_run,
    AVG(duration_seconds) / 60 AS avg_duration_minutes
FROM monitoring.costs
GROUP BY DATE(timestamp), pipeline
ORDER BY daily_cost DESC
```

This shows you:

- Which pipelines are most expensive
- Whether costs are trending up
- Whether duration improvements reduce cost

Step 12: Alerting Strategy — Making Alerts Actionable

Alert fatigue is real. Too many alerts and nobody listens.

Alert pyramid

```
          /\
         /  \
        /INFO\         (noisy, most teams mute these)
       /______\
      /        \
     / WARNING  \      (medium priority, requires attention)
    /____________\
   /              \
  /    CRITICAL    \   (few, actionable, pages engineers)
 /__________________\
```

Alert design principles

1. Alert on business impact, not technical details

- Bad: “CPU usage is 85%”
- Good: “Revenue dashboard is 4 hours stale (SLA: 2 hours)”

2. Each alert should have a runbook

- When an alert fires, engineers should know what to do
- If there’s no runbook, don’t alert

3. Escalate on silence

- Alert fires but goes unacknowledged for 1 hour → escalate
- Escalate again at 2 hours → page VP

4. Group related alerts

- If 5 pipelines all fail from the same source → one alert explaining root cause
- Not 5 separate alerts

Example alert definitions

```yaml
alerts:
  - name: "SLA_FRESHNESS_BREACH"
    condition: "freshness_hours > sla_freshness_hours"
    severity: "CRITICAL"
    action: "trigger_auto_recovery_and_page_on_call"
    notify:
      - slack: "#data-incident"
      - pagerduty: "data-team-oncall"
    runbook: "https://wiki.company.com/data/sla-breach-runbook"

  - name: "DATA_QUALITY_DEGRADATION"
    condition: "error_count > 5 in last 1 hour"
    severity: "WARNING"
    action: "notify_and_log"
    notify:
      - slack: "#data-team"
    runbook: "https://wiki.company.com/data/quality-degradation-runbook"

  - name: "RECURRING_ERROR_PATTERN"
    condition: "same_error_type > 3 times in 24 hours"
    severity: "WARNING"
    action: "create_ticket_and_notify"
    notify:
      - slack: "#data-team"
      - jira: "create_ticket"
```

Step 13: The Observability Maturity Model

Where are you on the journey?



Level	Characteristics	Typical Issues
1. None	No monitoring at all	Discover issues from user complaints
2. Basic	Logs exist, but unstructured	Hard to debug; most time in investigation
3. Metrics	Dashboards exist, thresholds are set	Alert fatigue; high false positive rate
4. Structured	Logs are structured, traces exist	Good coverage of known issues; surprises still happen
5. Predictive	Anomalies detected early, SLA forecasting	Rare surprises; most issues caught before impact
6. Intelligent	Automatic root cause analysis, auto-recovery	Humans intervene rarely; system self-heals

Most mature data platforms are at Level 4–5. Level 6 is emerging.

Step 14: Interview Questions You Will Actually Be Asked at Level 5 (Production Readiness & Observability)

Question	What a strong answer should include
How do you know a pipeline is healthy?	Golden signals (freshness, completeness, latency, accuracy, reliability)
What is the difference between monitoring and observability?	Monitoring = measurement; observability = understanding why
Design an alerting strategy.	Business impact focus, escalation strategy, runbooks, reduce false positives
How would you detect a data quality issue automatically?	Anomaly detection, statistical methods, SLA thresholds
Describe distributed tracing across pipelines.	Trace IDs, propagation, root cause analysis of cascading failures
How do you forecast SLA breaches?	Historical duration analysis, percentile-based prediction, alert on predicted breach
What chaos experiments would you run?	Missing source data, slow dependencies, data quality degradation
How do you handle alert fatigue?	Alert on business impact not technical details, escalation strategy, runbooks
Explain your cost observability approach.	Cost per pipeline, cost trends, correlation with performance, optimization targets
Design an incident response process.	Detection, alerting, runbook execution, escalation, post-mortem

Step 15: Building a Production Observability System — The Implementation Path

You don’t build Level 5 (Production Readiness & Observability) all at once.

Month 1: Foundation

- Structured logging for all pipelines
- Golden signals metrics table
- Basic dashboard showing pipeline status
- Alert on job failure

Month 2: Detection

- Statistical anomaly detection for key metrics
- SLA freshness monitoring
- Data completeness checks
- Alert on SLA breaches

Month 3: Understanding

- Trace IDs across pipeline dependencies
- Runbooks for common failures
- Distributed tracing dashboard
- Root cause analysis capability

Month 4: Resilience

- Chaos experiments (monthly schedule)
- Auto-recovery for known issues
- Incident response runbooks
- Post-mortem process

Month 5+: Maturity

- Predictive alerting
- Cost optimization driven by observability
- Cross-team metrics standards
- Self-healing automation

Common Mistakes in Level 5 (Production Readiness & Observability)

Mistake	Why it hurts	Fix
Unstructured logging	Can't query logs when debugging	Structured logs with context fields
Too many alerts	Alert fatigue, nobody responds	Alert on business impact, high threshold
No runbooks	When alert fires, no one knows what to do	Runbook for every critical alert
Monitoring without understanding	You know something broke but not why	Add distributed tracing and context logging
No cost tracking	Cloud bill is a surprise	Track cost per pipeline, correlate with performance
Ignoring anomalies	Silent data quality failures	Implement statistical anomaly detection
No escalation strategy	Alerts go unnoticed	Escalate to different channels over time
Chaos experiments never run	You don't know how your system fails	Schedule chaos tests regularly
No incident response process	Chaos when things break	Documented runbooks and escalation
Observability data not queryable	Dashboard exists but you can't dig deeper	Store everything in queryable tables

Conclusion

Level 5 (Production Readiness & Observability) is where trust lives.

At Level 4 (Pipeline Engineering), you built pipelines that run.
At Level 5 (Production Readiness & Observability), you build systems you understand completely.

The difference is not in complexity. It is in **visibility**.

You can answer these questions immediately:

- Is my system healthy right now?
- When will it break?
- Why did it break?
- How do I fix it?
- Will it break the same way again?

That is what Level 5 (Production Readiness & Observability) delivers.

And once you have that visibility, the next question becomes:

Can I make the system faster and cheaper without losing reliability?

That is Level 6 (Performance & Cost Optimization).

Wait for the Next Detailed Blog

Next Monday, 11 AM IST
We will go deep into **Level 6 (Performance & Cost Optimization)**:

- Query optimization techniques
- Data layout and clustering strategies
- Spark tuneable and executor sizing
- Cost per query analysis
- Materialized views and caching strategies
- Partitioning vs clustering tradeoffs revisited
- Workload characterization
- Resource isolation and quotas
- Chargeback models
- Benchmarking and profiling

Databricks + Data Engineering + Spark + Python + Big Data +